**MeuAmparo**

# Seminário de desenvolvimento

Detecção de queda e pedido de ajuda rodando no simulador



Luis Gustavo Moda
Isabelly Vitoria

Programação de Microcontroladores (ARA4710)
Estácio Ribeirão Preto • 28/10/2026

# Onde estamos

## Pronto

- Detector de queda e máquina de estados, com testes
- Firmware: sensor, botão, LED, buzzer e visor
- Circuito e 4 cenários no Wokwi
- 11 requisitos, diagrama de blocos e fluxograma (a validar com a Casa do Vovô)

## Em andamento

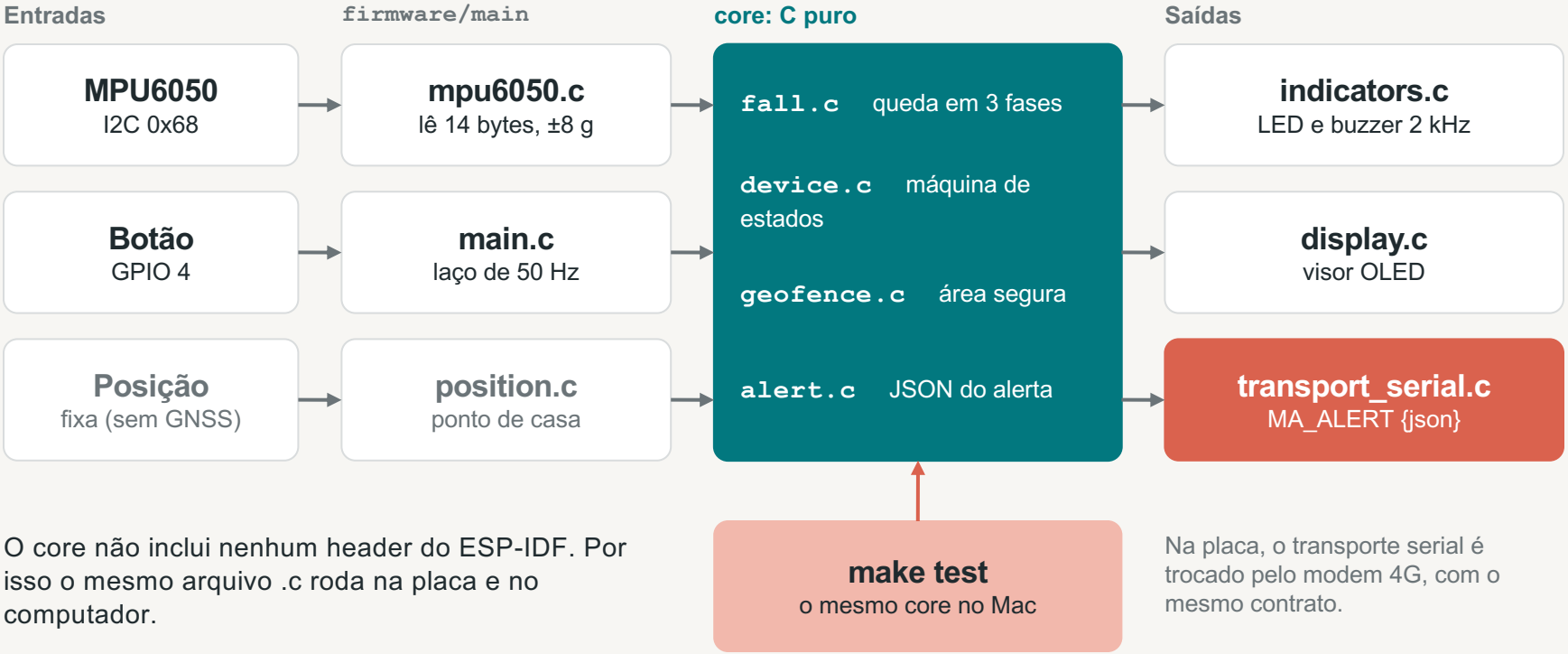
- Teste com dataset real (SisFall)
- Registro dos testes
- Roteiro 3 e relatos

## Pendente

- Validação com a Casa do Vovô: reunião em 03/11
- Lar Padre Euclides e Lar do Vovô Albano sem retorno

Backend e painel saíram do escopo: o esforço foi para o código embarcado

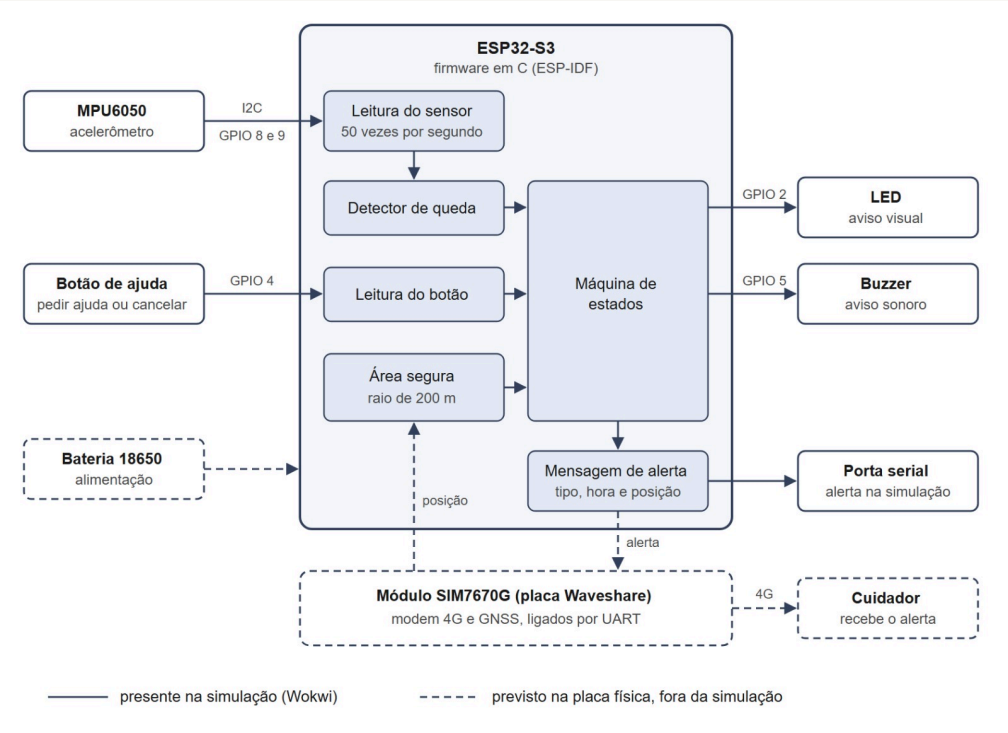
# Arquitetura: do sensor ao alerta




O core não inclui nenhum header do ESP-IDF. Por isso o mesmo arquivo .c roda na placa e no computador.

Na placa, o transporte serial é trocado pelo modem 4G, com o mesmo contrato.

# Diagrama de blocos do hardware



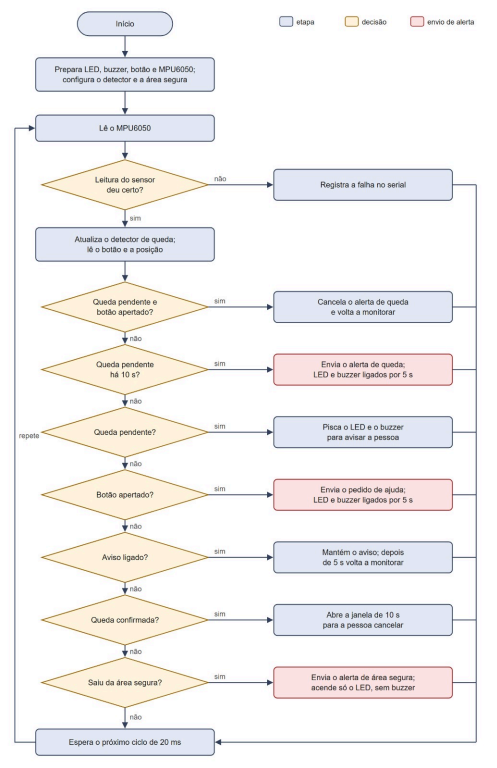

Linha cheia: presente na simulação. Tracejado: previsto na placa física (modem 4G e GNSS do SIM7670G, bateria 18650).

O alerta sai pela porta serial na simulação; na placa, sairia pelo modem 4G até o cuidador.

O visor OLED entrou no firmware depois deste diagrama (28/10).

Diagrama: Isabelly Vitoria

# Fluxograma do firmware




## Um ciclo a cada 20 ms

lê o sensor, atualiza o detector, lê o botão e a posição

## Decisões em ordem de prioridade

cancelamento primeiro, depois alerta de queda, aviso, botão e área segura

## Três saídas vermelhas

alerta de queda, pedido de ajuda e saída da área segura

## Falha de leitura não trava

registra no serial e segue para o próximo ciclo

Diagrama: Isabelly Vitoria

# Como as pastas estão organizadas

**meuamparo/**

**src/meuamparo/** o que roda

**docs/**
**material-disciplina/**
**projeto/**

o que vai para a faculdade

**Makefile**
make sim, make test

**core/**
**include/meuamparo/**
fall.h
device.h
geofence.h
alert.h
imu.h, types.h
**src/**
os .c de cada um

**firmware/**
**main/**
main.c
mpu6050.c
display.c
indicators.c
position.c
transport_serial.c
i2c_bus.c
board.h
**components/**
meuamparo_core

**simulator/**
**wokwi/**
diagram.json
wokwi.toml
**scenarios/**
botao.yaml
queda.yaml
queda-cancelada
pulo.yaml

**test/**
test_fall.c
test_device.c
test_geofence.c
test_imu.c
ma_test.h

Pastas e arquivos reais de src/meuamparo em 28/10/2026

ver no GitHub ↗

Arquitetura

# O core decide, o firmware executa

## `core/` decide

C puro, sem nenhum header do ESP-IDF: detector de queda, máquina de estados, área segura e formato do alerta.

## `firmware/` executa

ESP-IDF: lê o MPU6050 por I2C e o botão, liga LED, buzzer e visor, envia o alerta. Sem mock nem cenário.

**`simulator/`**
a bancada: circuito e cenários do Wokwi

**`test/`**
a prova: o core testado no computador

### Por que dividimos

- Medir o detector contra centenas de registros em segundos, no Mac, sem gravar a placa
- O mesmo .c compila nos dois lados: components/meuamparo_core aponta para core/src
- Trocar placa, visor ou transporte não mexe na lógica

# O que usamos



**C11 + ESP-IDF v5.5.5**
framework oficial da Espressif para o ESP32



**ESP32-S3**
devkit no simulador; Waveshare SIM7670G no projeto



**wokwi-cli 0.26.1**
roda o firmware real no Wokwi, sem tela, nos cenários



**Wokwi for VS Code 4.1.0**
o mesmo circuito na tela, para ver e interagir

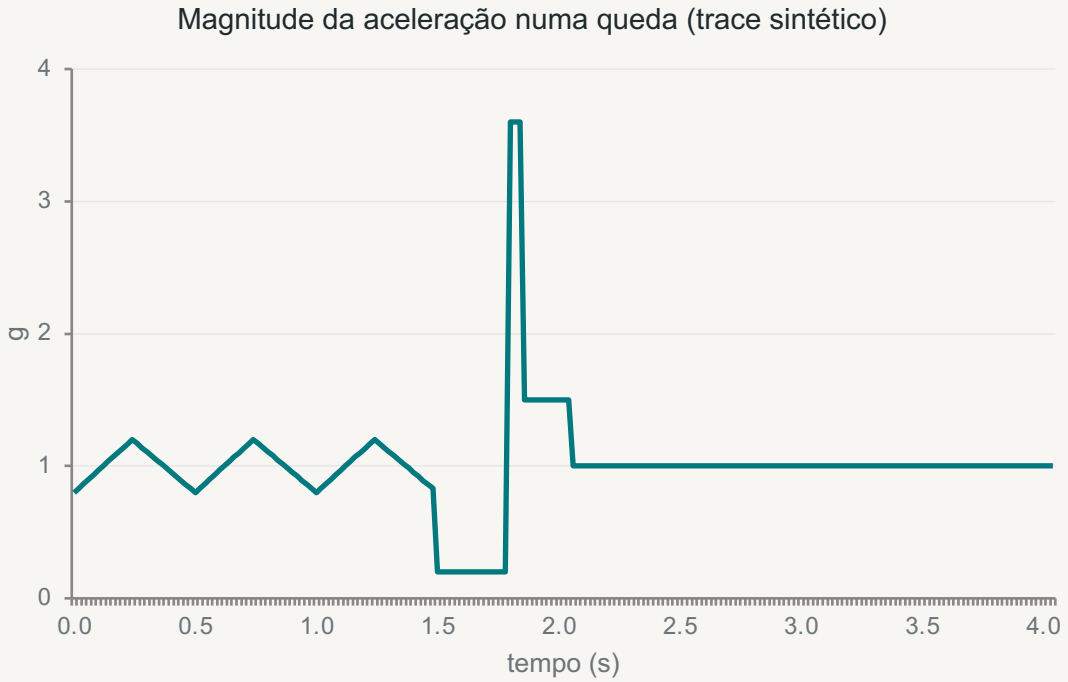


**Make**
um comando para testar, compilar e simular



**GitHub + GPL v3**
issues e quadro do projeto; código aberto

# Detector de queda em três fases




**1 Queda livre**
abaixo de 0,5 g por 60 ms

**2 Impacto**
acima de 2,5 g em até 1 s

**3 Imobilidade**
perto de 1 g por 1,5 s

**Pulo também tem 1 e 2. A fase 3 separa queda de pulo**

# O código do detector

```
65  case MA_FALL_AWAIT_IMPACT:
66      if (mag >= c->impact_g) {
67          enter(d, MA_FALL_SETTLE, t);
68      } else if (elapsed > c->impact_window_ms) {
69          enter(d, MA_FALL_IDLE, t);
70      }
71      return false;
    …
79  case MA_FALL_STILLNESS:
80      /* Qualquer movimento cancela: a pessoa levantou ou nunca
caiu de fato. */
81      if (fabsf(mag - 1.0f) > c->stillness_tol_g) {
82          enter(d, MA_FALL_IDLE, t);
83          return false;
84      }
85      if (elapsed >= c->stillness_ms) {
86          enter(d, MA_FALL_IDLE, t);
87          return true;
88      }
89      return false;
```

core/src/fall.c, linhas 65–71 e 79–89

ver no GitHub ↗

**50 amostras por segundo**

cada uma passa por ma_fall_update()

**Cada fase tem tempo limite**

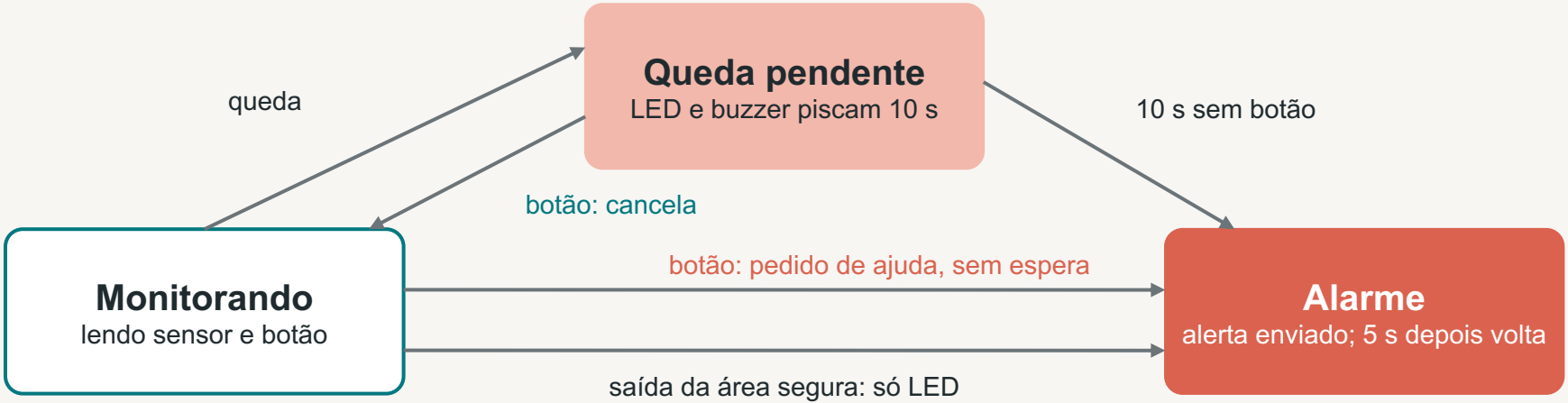
sem impacto em 1 s, volta ao início

**Devolve true uma única vez**

na amostra que confirma a queda

core/src/device.c

# O que o dispositivo faz depois




A janela de 10 s deixa a própria pessoa cancelar um alarme falso, sem depender de afinar limiar.

# A janela de cancelamento no código

```
40      if (in.button) {
41          raise_alarm(d, t, &out, MA_ALERT_PANIC, true);
42      } else if (in.fall) {
43          if (d->cfg.cancel_window_ms == 0) {
44              raise_alarm(d, t, &out, MA_ALERT_FALL, true);
45          } else {
46              d->state = MA_DEV_FALL_PENDING;
47              d->t_state = t;
48          }
49      } else if (in.geofence_exit) {
50          raise_alarm(d, t, &out, MA_ALERT_GEOFENCE, false);
51      }
52      break;
    …
54  case MA_DEV_FALL_PENDING:
55      if (in.button) {
56          d->state = MA_DEV_MONITORING;
57          d->t_state = t;
58          out.cancelled = true;
59      } else if (elapsed >= d->cfg.cancel_window_ms) {
60          raise_alarm(d, t, &out, MA_ALERT_FALL, true);
61      }
62      break;
```

core/src/device.c, linhas 40–52 e 54–62

ver no GitHub ↗

## Linhas 40 a 41

botão em monitoramento: alerta de ajuda na hora

## Linhas 42 a 48

queda abre a janela; com janela zero, alerta direto

## Linhas 55 a 61

botão na janela cancela; sem botão, alerta de queda

firmware/main/main.c

# Firmware fino: lê, entrega ao core, obedece

```
220  ma_device_input_t in = {
221      .fall = ma_fall_update(&fall, &s),
222      .button = button_pressed_edge(),
223  };
224  ma_geo_point_t pos;
225  if (ma_position_get(&pos)) {
226      in.geofence_exit = ma_geofence_update(&fence, &pos);
227  }
228
229  const ma_device_output_t out = ma_device_step(&dev, t, in);
230  ma_indicators_set(out.led, out.buzzer);
     …
272      banner(what[out.kind], "mensagem com tipo, hora e posicao:");
273      send_alert(out.kind, t);
274  }
```

firmware/main/main.c, linhas 220–230 e 272–274

ver no GitHub ↗

**mpu6050.c**

I2C, faixa de ±8 g

**indicators.c**

LED e buzzer por PWM

**display.c**

visor OLED 128x64

**position.c**

posição fixa, sem GNSS

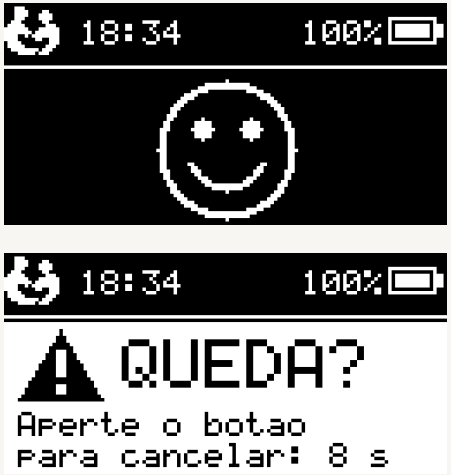
**transport_serial.c**

alerta no serial

A cada 20 ms: lê sensor e botão, chama o core, liga LED, buzzer e visor, envia o alerta

# Circuito e pinos

| Componente | Função | Ligação no ESP32-S3 |
|---|---|---|
| MPU6050 | aceleração e rotação | I2C 0x68: SDA 8, SCL 9 |
| OLED SSD1306 | visor 128x64 | I2C 0x3C: mesmo barramento |
| Botão | pedido de ajuda e cancelamento | GPIO 4, pull-up |
| Buzzer | aviso sonoro, PWM 2 kHz | GPIO 5 |
| LED + 330 Ω | aviso visual | GPIO 2 |
| Potenciômetro | faz o papel da bateria | GPIO 1, ADC1 canal 0 |




Visor no Wokwi: monitorando e na janela de cancelamento

**O OLED substitui o visor touch de 1,69" previsto, que o Wokwi não tem.** Os pinos são do devkit: na Waveshare ESP32-S3-SIM7670G parte deles é do modem, e a pinagem precisa ser revista antes de montar.

Pinos em firmware/main/board.h; circuito em simulator/wokwi/diagram.json

# Um comando: make sim

```
$ make help
  help           Lista os alvos disponíveis
  test           Compila e roda os testes de host do core
  setup          Instala ESP-IDF e wokwi-cli, pulando o que já existe
  firmware       Compila o firmware ESP-IDF (carrega o ambiente sozinho)
  sim            Roda todos os cenários do Wokwi headless (queda, botão, pulo...)
  sim-queda      Cenário: queda, janela de 10 s e alerta com posição
  sim-cancelada  Cenário: queda cancelada pelo botão dentro da janela
  sim-botao      Cenário: pedido de ajuda pelo botão
  sim-pulo       Cenário: pulo, que não pode gerar alerta
  web            Gera o .bin para rodar no wokwi.com e mostra o passo a passo
  clean          Apaga os binários gerados em build/ e no firmware
```

## make sim, em ordem:

**1** carrega o ESP-IDF, se a shell não carregou

**2** lê o token do Wokwi de ~/.config/wokwi/token

**3** compila o firmware com idf.py build

**4** roda os 4 cenários; o primeiro que falhar para tudo

make sim-queda, sim-cancelada, sim-botao e sim-pulo rodam um cenário só, com o log inteiro na tela

src/meuamparo/Makefile

# O alvo sim por dentro

```
102  sim: firmware ## Roda todos os cenários do Wokwi headless (queda, botão, pulo...)
103      @command -v wokwi-cli >/dev/null 2>&1 || { \
104          echo "wokwi-cli não encontrado. Instale com:"; \
105          echo "  make setup"; \
106          exit 1; }
107      @$(WOKWI_TOKEN); \
108      for s in $(SCENARIOS); do \
109          echo "== $$s"; \
110          wokwi-cli --quiet --timeout 40000 --scenario scenarios/$$(basename $$s) $(WOKWI_DIR) || exit 1; \
111      done
112      @echo "ok: todos os cenários passaram"
```

Makefile, linhas 102–112

ver no GitHub ↗

**`sim: firmware`**

depende do alvo firmware: sempre testa o binário recém-compilado

**`$(WOKWI_TOKEN)`**

lê o token do arquivo, sem ele aparecer no repositório

**`|| exit 1`**

o primeiro cenário que falhar interrompe tudo

# Cenários: o sistema inteiro testado sozinho

```yaml
steps:
  - wait-serial: 'monitorando'
  - delay: 1000ms
  # queda livre: o sensor quase não sente aceleração
  - set-control: { part-id: imu1, name: accelZ, value: 0.1 }
  - delay: 300ms
  # impacto contra o chão
  - set-control: { part-id: imu1, name: accelZ, value: 4 }
  - delay: 60ms
  # caída de lado, parada: a gravidade agora aparece no eixo X
  - set-control: { part-id: imu1, name: accelZ, value: 0 }
  - set-control: { part-id: imu1, name: accelX, value: 1 }
  - wait-serial: 'queda confirmada'
  # ninguém cancela: depois da janela o alerta sai com a posição
  - wait-serial: 'MA_ALERT {"device":"ma-01","kind":"fall"'
  - expect-pin: { part-id: esp, name: 2, value: 1 }
```

simulator/wokwi/scenarios/queda.yaml, linhas 7–22

ver no GitHub

**`set-control`**

mexe no sensor ou aperta o botão

**`wait-serial`**

espera uma linha no serial do firmware

**`expect-pin`**

confere o nível de um pino, como o LED

**4/4** cenários passaram

botão, queda, queda cancelada, pulo

# Ver e interagir: a extensão do Wokwi

1. **Abrir a pasta**
   `code src/meuamparo/simulator/wokwi`

2. **Licença, só na primeira vez**
   `F1 → Wokwi: Request a new License`

3. **Rodar**
   `F1 → Wokwi: Start Simulator`

4. **Interagir**
   clicar no botão, mexer nos sliders do MPU6050, ler o visor e o serial

```
3  [wokwi]
4  version = 1
5  firmware = "../../firmware/build/meuamparo.bin"
6  elf = "../../firmware/build/meuamparo.elf"
```

simulator/wokwi/wokwi.toml, linhas 3–6 ver no GitHub ↗

O wokwi.toml aponta para o mesmo binário que o make sim usa. O que se vê na tela é o firmware que passou nos testes.

Sem VS Code: make web gera um .bin para subir no wokwi.com.

# Testes em duas camadas

## make test, no computador

**50/50**

quedas fortes detectadas

**22/50**

quedas amortecidas

0 alarme falso em andar, correr, pular, sentar com força, deitar e cair e levantar

**Traces escritos pelo grupo: não medem a meta de 90%. A medição usará o SisFall.**

## make sim, no Wokwi

**4/4**

cenários passaram

**Botão:** alerta panic e LED aceso

**Queda:** janela de 10 s e alerta fall

**Queda cancelada:** botão na janela, sem alerta

**Pulo:** volta a monitorar, sem alerta

Por que isso importa

# O que esse jeito de trabalhar nos dá

## 1 comando

**make sim reproduz tudo**

compila e testa o sistema inteiro em qualquer máquina com as ferramentas

## 1 código

**o mesmo C no Mac e na placa**

o detector é afinado no computador, sem gravar a placa a cada ajuste

## 4 cenários

**regressão do sistema integrado**

mudou um limiar e quebrou a queda? o make sim acusa

## Limites claros

**o que não está provado**

traces sintéticos, sem hardware, GNSS fixo: dito em voz alta

# Demonstração ao vivo

**No terminal, na raiz do repositório**

| | |
|---|---|
| `make sim-queda` | queda, janela de 10 s e alerta com posição |
| `make sim-cancelada` | queda cancelada pelo botão dentro da janela |
| `make sim-botao` | pedido de ajuda pelo botão |
| `make sim-pulo` | pulo, que não pode gerar alerta |



`QUEDA LIVRE → IMPACTO → PESSOA PARADA, DEITADA → QUEDA DETECTADA → ALERTA ENVIADO`

**Wokwi no VS Code:** circuito ao vivo, botão e visor

Relato

# Dificuldades e problemas

### Instituições

Só a Casa do Vovô aceitou: reunião em 03/11. Lar Padre Euclides não respondeu nem atendeu; Lar do Vovô Albano, sem retorno da assistente social

### Requisitos

11 requisitos escritos antes da escuta, a partir dos objetivos, concluídos com atraso no fim de outubro; serão validados na reunião

### Tempo e custo

16/09: não comprar a placa, sem tempo hábil. Waveshare estimada em R$ 597 a R$ 622 (set/2026)

### Wokwi

primeira vez usando: token de CI, licença da extensão, cenários

### Só simulação

Wokwi sem GNSS nem modem 4G: posição fixa e alerta no serial

### Ambiente

queda difícil de provocar com slider; atualização do Python quebrou o ESP-IDF (resolvido no Makefile)

Valores de setembro de 2026: estimativa, não preço atual

# O que cada um fez e aprendeu

## Gustavo

- Arquitetura do código
- Detector, máquina de estados e firmware
- Circuito, visor e cenários no Wokwi
- Repositório, Makefile e modelo do case

***O que eu aprendi?***

## Isa

- Contato com as três instituições; reunião marcada na Casa do Vovô (03/11)
- Escopo e 11 requisitos funcionais, cada um com forma de verificação
- Diagrama de blocos e fluxograma
- Cronograma e slides

***O que eu aprendi?***

Até 25/11

# Limites e próximos passos

## O que a simulação não prova

- Autonomia de 8 horas
- Conforto e fixação no corpo
- Alarme falso no dia a dia
- Cobertura do 4G e precisão do GPS

## Até a entrega final

1. Reunião com a Casa do Vovô (03/11): validar requisitos e diagramas
2. Medir o detector com o SisFall
3. GPS: hoje a posição é fixa; falta o GNSS (driver ou chip simulado)
4. Avaliação de reação com a instituição
5. Roteiro 3 e relatos
6. Seminário de avaliação, 25/11